Rapport - Optimisation d'un système de préparation de livraisons

Langage : Python

J'ai choisi Python pour trois raisons :

- c'est le langage avec lequel je suis le plus à l'aise, et celui que j'ai toujours utilisé pour écrire des algorithmes
- sa syntaxe est concise, ce qui rend les algorithmes plus lisibles : le code reste proche de l'idée de l'algorithme
- la mémoire est gérée automatiquement (pas de `malloc`/`free`), ce qui me permet de me concentrer sur les choix algorithmiques plutôt que sur la gestion des pointeurs et des allocations.

Version utilisée : Python 3.14 (venv).

Choix algorithmiques

1. Stockage des colis

Élément	
Problème	Il faut pouvoir stocker beaucoup de colis (jusqu'à 100 000) et retrouver vite un colis avec son ID.
Solution	J'utilise un dictionnaire Python (une table de hachage), avec l'ID comme clé et le colis comme valeur.
Temps	Ajout : O(1) en moyenne. Recherche par ID : O(1) en moyenne. Affichage : O(n).
Espace	O(n)
Alternative	Une simple liste. L'ajout est aussi en O(1), mais pour retrouver un colis il faut parcourir la liste, donc O(n).
Justification	On cherche souvent un colis par son ID. Avec 100 000 colis, une liste peut demander 100 000 comparaisons pour une seule recherche, alors que le dictionnaire va directement au bon endroit. En plus, le dictionnaire garde l'ordre d'ajout et ça permet de voir tout de suite si un ID existe déjà.

2. Tri des colis (par poids, valeur ou priorité)

Élément	
Problème	Il faut pouvoir afficher les colis triés selon différents critères.
Solution	Un tri fusion. On coupe la liste en deux, on trie chaque moitié, puis on fusionne les deux moitiés triées. Le critère est passé en paramètre, donc la même fonction sert pour le poids,

Élément	
	la valeur et la priorité, dans les deux sens.
Temps	O(n log n), même dans le pire cas.
Espace	O(n) pour les listes temporaires de la fusion, plus O(log n) pour la récursion.
Alternative	Le tri rapide (quicksort) est aussi en O(n log n) en moyenne et ne prend pas de mémoire en plus, mais il peut tomber à O(n^2) dans le pire cas et il n'est pas stable. Le tri par insertion est en O(n^2), donc trop lent pour 100 000 colis.
Justification	Le temps est garanti peu importe les données. Le tri est stable, donc deux colis avec le même poids restent dans le même ordre qu'avant. Et la récursion reste peu profonde (environ 17 niveaux pour 100 000 colis), donc pas de souci avec la limite de récursion de Python.

3. Chargement du camion

Élément	
Problème	Il faut choisir quels colis mettre dans le camion pour avoir la plus grande valeur possible sans dépasser sa capacité C. Un colis est soit pris en entier, soit laissé. C'est le problème du sac à dos 0/1.
Solution	De la programmation dynamique. `dp[w]` contient la meilleure valeur qu'on peut avoir avec w kg. Pour chaque colis, et pour chaque w de C jusqu'au poids du colis, on regarde si c'est mieux de le prendre ou non : `dp[w] = max(dp[w], dp[w - poids] + valeur)`. On parcourt w à l'envers pour ne pas prendre deux fois le même colis. Pour savoir à la fin quels colis ont été pris, on garde pour chaque colis un tableau qui note où il a servi, et on remonte depuis C.
Temps	`O(n*C)`
Espace	O(C) pour `dp`, plus `n*C` octets pour retrouver les colis choisis.
Alternative	La force brute teste toutes les combinaisons, donc O(2^n). Elle trouve la meilleure solution, mais au-delà d'une trentaine de colis c'est beaucoup trop long. Le glouton trie les colis par rapport valeur/poids et les prend dans l'ordre tant qu'ils rentrent. C'est rapide, en O(n log n), mais ça ne donne pas toujours la meilleure solution. Par exemple avec A (8 kg, 20), B (6 kg, 25), C (5 kg, 18), D (4 kg, 14) et un camion de 9 kg, le glouton prend juste B pour une valeur de 25, alors que C + D donnent 32.
Justification	La programmation dynamique trouve toujours la meilleure solution. Pour 100 000 colis et un camion de 50 kg, ça prend environ 0,1 s. Par contre, le temps dépend de la capacité : avec un très gros camion ça deviendrait lent. Il faut aussi que les poids soient des entiers. J'ai gardé le glouton pour comparer les résultats.

Questions de l'étape 4

Pourquoi avez-vous choisi cette stratégie ?

On cherche la meilleure valeur possible, pas juste un chargement qui rentre. Le glouton est plus simple mais il se trompe sur certains cas (comme le camion de 9 kg plus haut), et la force brute est trop lente dès qu'il y a plus d'une trentaine de colis. La programmation dynamique est le seul des trois qui est à la fois toujours juste et assez rapide pour 100 000 colis avec un camion de taille normale.

La solution obtenue est-elle toujours optimale ?

Oui. Pour chaque colis et pour chaque capacité, l'algorithme compare les deux seules possibilités : prendre le colis ou ne pas le prendre. Si on le prend, le reste du camion (w - poids kg) est rempli avec la meilleure solution déjà calculée pour les colis d'avant. Comme aucun cas n'est oublié, `dp[C]` contient forcément la meilleure valeur. Je l'ai vérifié en comparant avec la force brute sur 300 cas au hasard (`test_dp_optimale` dans `tests.py`). S'il existe plusieurs chargements avec la même valeur maximale, l'algorithme en renvoie un seul.

Quelle est sa complexité temporelle ?

`O(n*C)`, avec n le nombre de colis et C la capacité en kg. Pour chaque colis, on parcourt toutes les capacités de C jusqu'à son poids. Pour 100 000 colis et 50 kg ça fait environ 5 millions d'opérations, soit 0,1 s. Par contre ce temps dépend de la valeur de C et pas seulement du nombre de colis : avec un camion de 10 000 kg on monterait à 1 milliard d'opérations.

Quelle est sa complexité spatiale ?

`O(n*C)`. Le tableau `dp` ne prend que C + 1 cases, mais pour retrouver les colis choisis je garde une ligne de C + 1 octets par colis. J'ai utilisé des `bytearray` (1 octet par case) plutôt que des listes Python (au moins 8 octets par case), ce qui fait environ 5 Mo pour 100 000 colis et 50 kg. Si on voulait seulement la valeur maximale sans la liste des colis, on pourrait descendre à O(C).

Auriez-vous pu utiliser une autre approche ?

Oui, plusieurs :

- la force brute, qui teste les 2^n combinaisons : toujours juste mais inutilisable au-delà d'une trentaine de colis
- le glouton par rapport valeur/poids : très rapide (O(n log n)) mais pas toujours optimal. Il pourrait être utile si le camion était énorme et qu'une solution approchée suffisait
- la même programmation dynamique écrite en récursif avec mémoïsation : même complexité, mais avec 100 000 colis on dépasserait la limite de récursion de Python, donc la version avec une boucle est plus adaptée

4. Prise en compte des priorités

Élément	
Problème	L'entreprise veut favoriser les colis urgents. La priorité va de 1 (la moins urgente) à x (la plus urgente), et il faut la combiner avec le poids et la valeur.
Solution	Je ne change pas l'algorithme du camion, je lui donne un score à la place de la valeur : `score = valeur * (1 + (priorite - 1) / (x - 1))`, avec x la plus grande priorité parmi les colis à charger. Un colis de priorité 1 garde sa valeur, le plus urgent compte

Élément	
	double, et entre les deux ça augmente régulièrement. Le poids est déjà pris en compte par la capacité du camion.
Temps	O(n) pour trouver x, puis `O(n*C)` pour le chargement, donc `O(n*C)` au total.
Espace	Pareil qu'avant, `O(n*C)`.
Alternative	Un bonus fixe par priorité (par exemple +20 pour la plus urgente), mais son effet dépend de l'ordre de grandeur des valeurs. Charger par paliers : d'abord les colis les plus urgents, puis on complète avec les suivants. Un urgent passe toujours avant, mais on peut perdre beaucoup de valeur. Fixer x une fois pour toutes (par exemple 5) au lieu de prendre le max.
Justification	Le bonus est proportionnel à la valeur, donc un colis urgent qui ne vaut presque rien ne passe pas forcément devant un colis très cher. Diviser par x - 1 fait que le plus urgent compte toujours double, que l'échelle aille jusqu'à 5 ou jusqu'à 100. J'ai pris x comme la priorité max des colis du jour parce que je vois la priorité comme relative : le plus urgent du jour est toujours le plus favorisé. Si on ajoute un colis encore plus urgent, les scores des autres baissent un peu. Si tous les colis ont la même priorité, ils sont tous multipliés pareil et le chargement ne change pas.

Exemple avec les colis de l'énoncé (x = 5) :

Colis	Poids	Valeur	Priorité	Score
A	8	20	5	40
B	6	25	1	25
C	5	18	4	31,5
D	4	14	3	21

Camion	Sans priorité	Avec priorité
9 kg	C + D, valeur 32	C + D, valeur 32
10 kg	B + D, valeur 39	C + D, valeur 32
12 kg	B + C, valeur 43	A + D, valeur 34
13 kg	B + C, valeur 43	A + C, valeur 38

Avec 12 kg par exemple, on perd 9 de valeur mais A, le colis le plus urgent, part. C'est le compromis voulu : la DP reste optimale, mais pour le score et plus pour la valeur seule.

Recherche et performances

Les mesures sont faites avec `perf.py`. Les colis sont générés au hasard (poids de 1 à 30 kg, valeur de 1 à 100, priorité de 1 à 5) avec une graine fixe, donc on retrouve les mêmes colis à chaque lancement. Chaque mesure est répétée et je garde le meilleur temps. Les temps exacts dépendent de la machine, ce qui compte c'est comment ils évoluent quand n est multiplié par 10.

1. Recherche par ID

Temps moyen pour une recherche :

n	Linéaire	Dichotomie	Dictionnaire
100	0,85 µs	0,33 µs	0,04 µs
1 000	7,41 µs	0,55 µs	0,04 µs
10 000	77,98 µs	0,76 µs	0,05 µs
100 000	2 501,66 µs	1,00 µs	0,05 µs

- **Complexité :** linéaire O(n), dichotomie O(log n), dictionnaire O(1) en moyenne.
- **Évolution :** la recherche linéaire est multipliée par 10 à chaque fois (et même par 30 au dernier palier, sûrement parce que la liste ne tient plus dans le cache du processeur). La dichotomie prend juste un peu plus de temps à chaque palier, parce qu'elle fait seulement environ 3 étapes de plus quand n est multiplié par 10. Le dictionnaire ne bouge pas.
- **Est-ce que le choix reste adapté ?** Oui. Le dictionnaire est le plus rapide à toutes les tailles et il ne ralentit pas. La dichotomie serait une bonne alternative, mais il faut garder la liste triée par ID, et ajouter un colis au milieu coûte O(n) parce qu'il faut décaler les suivants. La recherche linéaire n'est pas adaptée au-delà de quelques milliers de colis.

2. Tri pour l'affichage

n	`tri_fusion`	`sorted()`
100	0,1 ms	0,0 ms
1 000	1,1 ms	0,0 ms
10 000	14,6 ms	0,8 ms
100 000	196,0 ms	13,5 ms

- **Complexité :** O(n log n) pour les deux.
- **Évolution :** le temps est multiplié par environ 13 quand n est multiplié par 10, ce qui correspond bien à du n log n (un peu plus que 10). `sorted()` a la même complexité mais il est environ 15 fois plus rapide, parce qu'il est écrit en C et qu'il profite des parties déjà triées (Timsort).
- **Est-ce que le choix reste adapté ?** Oui, 0,2 s pour 100 000 colis c'est acceptable pour un affichage. Le vrai défaut, c'est qu'on refait le tri à chaque affichage.
- **Amélioration :** garder les listes triées en mémoire et ne les recalculer que quand on ajoute un colis. On gagne du temps mais ça prend plus de mémoire (une liste de plus par critère de tri).

3. Temps de chargement du camion

Avec un camion de 50 kg, quand n augmente :

n	DP	Glouton	Valeur DP	Valeur glouton
100	0,2 ms	0,2 ms	1 073	1 064

n	DP	Glouton	Valeur DP	Valeur glouton
1 000	1,8 ms	2,8 ms	2 686	2 686
10 000	16,9 ms	35,9 ms	4 694	4 694
100 000	167,7 ms	477,0 ms	4 978	4 978

Avec 10 000 colis, quand la capacité C augmente :

C	DP	Glouton	Mémoire du tableau des choix	Valeur DP	Valeur glouton
50 kg	0,02 s	37,2 ms	0,5 Mo	4 694	4 694
500 kg	0,26 s	37,9 ms	5,0 Mo	26 443	26 443
5 000 kg	3,08 s	36,2 ms	50,0 Mo	97 495	97 495

- **Complexité :** `O(n*C)` en temps et en mémoire pour la DP, O(n log n) pour le glouton.
- **Évolution :** quand n est multiplié par 10, la DP est multipliée par 10. Mais quand C est multiplié par 10, elle est aussi multipliée par 10, temps et mémoire compris, alors que le glouton ne bouge pas. C'est ce que veut dire pseudo-polynomial : le coût dépend de la taille du camion et pas seulement du nombre de colis. Avec un petit camion, la DP est même plus rapide que le glouton, parce que le glouton doit trier tous les colis.
- **Est-ce que le choix reste adapté ?** Oui pour des camions de taille normale, même avec 100 000 colis. Par contre avec 100 000 colis et un camion de 5 000 kg, on passerait à environ 30 s et 500 Mo, ce qui devient trop. On remarque aussi que sur des données aléatoires avec beaucoup de colis, le glouton trouve souvent la même valeur que la DP, parce qu'il y a beaucoup de petits colis qui comblent les trous. La différence se voit surtout avec peu de colis (1 073 contre 1 064 pour n = 100) ou sur des cas faits exprès comme le camion de 9 kg de l'énoncé.
- **Améliorations possibles :**
  - retirer d'avance les colis plus lourds que le camion, ils ne servent à rien dans la boucle
  - compter les poids par tranches (par exemple par 5 kg) pour diviser C, en acceptant un résultat un peu moins précis
  - utiliser le glouton quand C est très grand, parce que dans ces cas il est souvent très proche de l'optimal
  - si on veut seulement la valeur maximale et pas la liste des colis, on peut ne garder que `dp` et descendre à O(C) en mémoire